

Instalación de Arch Linux



Félix J. Marcelo Wirnitzer

- 1 Comprobamos si la ISO tiene más de un mes. Si es así, hay que descargar la última. Siempre se publica una nueva cada primero de mes.
- 2 Ya tenemos la ISO en `/home/otros/ISOs/archlinux-2024.02.01-x86_64.iso`
- 3 Y tostamos...

- 1 Comprobamos si la ISO tiene más de un mes. Si es así, hay que descargar la última. Siempre se publica una nueva cada primero de mes.
- 2 Ya tenemos la ISO en `/home/otros/ISOs/archlinux-2024.02.01-x86_64.iso`
- 3 Y tostamos...

- 1 Comprobamos si la ISO tiene más de un mes. Si es así, hay que descargar la última. Siempre se publica una nueva cada primero de mes.
- 2 Ya tenemos la ISO en `/home/otros/ISOs/archlinux-2024.02.01-x86_64.iso`
- 3 Y tostamos...

Establecemos el teclado en español

`loadkeys es`



```
ls /sys/firmware/efi/efivars
```



Comprobamos qué tipo de arranque tiene la placa

```
ls /sys/firmware/efi/efivars
```

Si no vemos nada tras la ejecución, entonces es un arranque **BIOS**.
En este caso deberemos instalar el paquete `grub-bios`.



Comprobamos la conexión a Internet

```
ping -c 4 google.es
```

Nota

En caso de que sea un portátil o un ordenador con conexión inalámbrica, se deberá usar la aplicación `iwctl`.

Comprobamos la conexión a Internet

```
ping -c 4 google.es
```

Nota

En caso de que sea un portátil o un ordenador con conexión inalámbrica, se deberá usar la aplicación `iwctl`.

Para proceder al particionado

Si se trata de un disco duro mecánico o una unidad SSD

```
cfdisk /dev/sda
```



Para proceder al particionado

Pero si se trata de una unidad M.2

```
cfdisk /dev/nvme0n1
```



Hay que tener en cuenta si el disco está particionado como **MBR** o como **GPT**

	MBR	GPT
Partición BIOS	No hace falta	Sí, en formato FAT32
Partición UEFI	No hace falta	Sí, en formato FAT32

Ejemplo de particionado MBR

```
/dev/sda1  /          ext4  mkfs.ext4 /dev/sda1  
/dev/sda2  swap        swap  mkswap /dev/sda2  
/dev/sda3  /home       ext4  mkfs.ext4 /dev/sda3
```

Aquí no importa si el arranque es de tipo BIOS o UEFI.

Ejemplo de particionado GPT con BIOS

```
/dev/sda1  BIOS    fat32  mkfs.fat -F32 /dev/sda1  
/dev/sda2  /          ext4   mkfs.ext4 /dev/sda1  
/dev/sda3  swap       swap   mkswap /dev/sda2  
/dev/sda4  /home     ext4   mkfs.ext4 /dev/sda3
```

Es recomendable que la partición BIOS sea el primero de todos.

Ejemplo de particionado GPT con EFI

```
/dev/sda1  EFI      fat32  mkfs.fat -F32 /dev/sda1  
/dev/sda2  /          ext4   mkfs.ext4 /dev/sda1  
/dev/sda3  swap        swap   mkswap /dev/sda2  
/dev/sda4  /home       ext4   mkfs.ext4 /dev/sda3
```

Es recomendable que la partición EFI sea el primero de todos.

Sabiendo que `/dev/sda2` será la partición raíz:

```
mount /dev/sda2 /mnt
```

Si es una instalación en un disco GPT con una partición BIOS o UEFI habrá que hacer

```
mount --mkdir /dev/sda1 /mnt/boot
```


Sabiendo que `/dev/sda2` será la partición raíz:

```
mount /dev/sda2 /mnt
```

Si es una instalación en un disco GPT con una partición BIOS o UEFI habrá que hacer

```
mount --mkdir /dev/sda1 /mnt/boot
```



Montamos la partición *swap*

```
swapon /dev/sda3
```

La partición *home* se montará más adelante.

Montamos la partición *swap*

```
swapon /dev/sda3
```

La partición *home* se montará más adelante.

Primero actualizamos el catálogo de paquetes

```
pacman -Sy
```



Instalación con BIOS: `grub-bios`

```
pacstrap -K /mnt base base-devel linux linux-firmware sudo ntfs-3g networkmanager curl aria2 zsh neovim \  
grub-bios sddm plasma openssh git
```



Instalación con UEFI: `grub` y `efibootmgr`

```
pacstrap -K /mnt base base-devel linux linux-firmware sudo ntfs-3g networkmanager curl aria2 zsh neovim \  
grub efibootmgr sddm plasma openssh git
```



Explicación de los paquetes instalados

- base:** Conjunto mínimo de paquetes para definir una instalación básica de Arch Linux.
- base-devel:** Conjunto de paquetes necesarios para crear software a partir del código fuente.
- linux:** El propio *kernel*.
- linux-firmware:** Controladores para hardware común.
- sudo:** Quieres ejecutar comandos como root, ¿verdad?
- ntfs-3g:** Controlador del sistema de archivos NTFS y utilidades necesarias para trabajar con unidades NTFS.
- networkmanager:** Facilita detección y configuración para que los sistemas se conecten automáticamente a las redes.
- curl:** Herramienta de línea de comandos y una biblioteca para transferir datos mediante URL.
- aria2:** aria2 es una ligera utilidad de descarga multiprotocolo y multifuente.
- zsh:** *Shell* potente que funciona como shell interactivo y como intérprete de lenguajes de scripting.
- neovim:** Editor de texto plano de gran potencia.
- grub:** Gestor multiarranque.
- openssh:** Protocolo de red para iniciar una sesión en una máquina remota.
- git:** Programa para control de versiones.



Antes de generar el fichero `fstab`, debemos montar la partición de *home*

Si es una instalación BIOS:

```
mount /dev/sda3 /mnt/home
```



Antes de generar el fichero `fstab`, debemos montar la partición de *home*

Si es una instalación UEFI:

```
mount /dev/sda4 /mnt/home
```



Generación de fstab

```
genfstab -U /mnt >> /mnt/etc/fstab
```

Comprobamos que se ha generado correctamente

```
cat /mnt/etc/fstab
```

Generación de fstab

```
genfstab -U /mnt >> /mnt/etc/fstab
```

Comprobamos que se ha generado correctamente

```
cat /mnt/etc/fstab
```

Y entramos en la partición `/mnt`

```
arch-chroot /mnt
```



Establecemos la zona horaria que es **Atlántico/Canarias**

```
ln -sf /usr/share/zoneinfo/Atlantic/Canary /etc/localtime
```

A continuación ejecutamos

```
hwclock --systohc
```

Establecemos la zona horaria que es **Atlántico/Canarias**

```
ln -sf /usr/share/zoneinfo/Atlantic/Canary /etc/localtime
```

A continuación ejecutamos

```
hwclock --systohc
```

Editamos con nano `/etc/locale.gen`

Aquí configuramos el idioma

```
nano /etc/locale.gen
```

Editamos con nano `/etc/locale.gen`

Aquí configuramos el idioma

```
nano /etc/locale.gen
```

Descomentamos la línea que contenga `es_ES.UTF-8 UTF-8`

```
#es_EC.UTF-8 UTF-8  
#es_EC ISO-8859-1  
es_ES.UTF-8 UTF-8  
#es_ES ISO-8859-1  
#es_ES@euro ISO-8859-15
```



Editamos con nano `/etc/locale.gen`

Aquí configuramos el idioma

```
nano /etc/locale.gen
```

Descomentamos la línea que contenga `es_ES.UTF-8 UTF-8`

```
#es_EC.UTF-8 UTF-8
#es_EC ISO-8859-1
es_ES.UTF-8 UTF-8
#es_ES ISO-8859-1
#es_ES@euro ISO-8859-15
```

Y tras guardar:

```
locale-gen
```



Para establecer el idioma por defecto:

```
echo 'LANG=es_ES.UTF-8' > /etc/locale.conf
```

```
echo 'KEYMAP=es' > /etc/vconsole.conf
```

Le damos nombre a nuestro ordenador

```
echo Lusitania > /etc/hostname
```



Añadimos las siguientes líneas en `/etc/hosts`

```
127.0.0.1  localhost
::1        localhost
127.0.1.1  Lusitania
```



Habilitamos NetworkManager para su arranque automático

```
systemctl enable NetworkManager
```

Establecemos la contraseña del usuario *root*

```
passwd
```



Seguiremos los siguientes pasos para la creación de nuestro usuario **viriato** con su contraseña **iberia**.

Creamos el grupo de **viriato**.

```
groupadd viriato
```

Creamos el usuario.

```
useradd viriato -m -g viriato
```

Con el parámetro `-m` indicamos que debe crear el directorio `/home/viriato` y `-g` para indicar el grupo principal del usuario.

Le ponemos al usuario una contraseña.

```
passwd viriato
```



Seguiremos los siguientes pasos para la creación de nuestro usuario **viriato** con su contraseña **iberia**.

Creamos el grupo de **viriato**.

```
groupadd viriato
```

Creamos el usuario.

```
useradd viriato -m -g viriato
```

Con el parámetro **-m** indicamos que debe crear el directorio **/home/viriato** y **-g** para indicar el grupo principal del usuario.

Le ponemos al usuario una contraseña.

```
passwd viriato
```


Seguiremos los siguientes pasos para la creación de nuestro usuario **viriato** con su contraseña **iberia**.

Creamos el grupo de **viriato**.

```
groupadd viriato
```

Creamos el usuario.

```
useradd viriato -m -g viriato
```

Con el parámetro **-m** indicamos que debe crear el directorio **/home/viriato** y **-g** para indicar el grupo principal del usuario.

Le ponemos al usuario una contraseña.

```
passwd viriato
```



Queremos darle a **viriato** privilegios de tipo *sudo*.

Para ellos debemos editar el fichero **/etc/sudoers**

```
nano /etc/sudoers
```

Y añadimos la siguiente línea

```
viriato ALL=(ALL:ALL) NOPASSWD:ALL
```

Guardamos y cerramos el fichero.

En Arch Linux, las actualizaciones de microcódigo están disponibles a través de paquetes oficiales que todo usuario debería instalar en sus sistemas.

Los fabricantes de procesadores como Intel y AMD lanzan a menudo actualizaciones de estabilidad y seguridad para el procesador. Estas actualizaciones son cruciales para la estabilidad del sistema.

En el caso de Intel

```
pacman -S intel-ucode
```

En Arch Linux, las actualizaciones de microcódigo están disponibles a través de paquetes oficiales que todo usuario debería instalar en sus sistemas.

Los fabricantes de procesadores como Intel y AMD lanzan a menudo actualizaciones de estabilidad y seguridad para el procesador. Estas actualizaciones son cruciales para la estabilidad del sistema.

En el caso de AMD

```
pacman -S amd-ucode
```

Configuramos GRUB, el cargador de arranque. Debemos tener en cuenta si la placa tiene BIOS o UEFI como vimos al principio.

Si se trata de un arranque BIOS con disco mecánico o una unidad SSD

```
grub-install /dev/sda
```



Configuramos GRUB, el cargador de arranque. Debemos tener en cuenta si la placa tiene BIOS o UEFI como vimos al principio.

Si se trata de un arranque BIOS con una unidad M.2

```
grub-install /dev/nvme0n1
```



Configuramos GRUB, el cargador de arranque. Debemos tener en cuenta si la placa tiene BIOS o UEFI como vimos al principio.

Si se trata de un arranque UEFI

```
grub-install --target=x86_64-efi --efi-directory=/boot --bootloader-id=grub
```

Configuramos GRUB, el cargador de arranque. Debemos tener en cuenta si la placa tiene BIOS o UEFI como vimos al principio.

Guardamos la configuración:

```
grub-mkconfig -o /boot/grub/grub.cfg
```

Y si no muestra un mensaje de error, podemos reiniciar comprobando que podemos arrancar Arch Linux.



El **Arch User Repository (AUR)** es un repositorio comunitario para usuarios de Arch.

Contiene descripciones de paquetes (PKGBUILDs) que permiten compilar un paquete desde el código fuente con makepkg y luego instalarlo mediante **pacman**.

El **AUR** se creó para organizar y compartir nuevos paquetes de la comunidad y para ayudar a acelerar la inclusión de paquetes populares en el repositorio extra.

La aplicación que usaremos para instalar paquetes de los repositorios de **AUR** será **yay**.



El **Arch User Repository (AUR)** es un repositorio comunitario para usuarios de Arch.

Contiene descripciones de paquetes (PKGBUILDS) que permiten compilar un paquete desde el código fuente con makepkg y luego instalarlo mediante **pacman**.

El **AUR** se creó para organizar y compartir nuevos paquetes de la comunidad y para ayudar a acelerar la inclusión de paquetes populares en el repositorio extra.

La aplicación que usaremos para instalar paquetes de los repositorios de **AUR** será **yay**.

El **Arch User Repository (AUR)** es un repositorio comunitario para usuarios de Arch.

Contiene descripciones de paquetes (PKGBUILDS) que permiten compilar un paquete desde el código fuente con makepkg y luego instalarlo mediante **pacman**.

El **AUR** se creó para organizar y compartir nuevos paquetes de la comunidad y para ayudar a acelerar la inclusión de paquetes populares en el repositorio extra.

La aplicación que usaremos para instalar paquetes de los repositorios de **AUR** será **yay**.



El **Arch User Repository (AUR)** es un repositorio comunitario para usuarios de Arch.

Contiene descripciones de paquetes (PKGBUILDs) que permiten compilar un paquete desde el código fuente con makepkg y luego instalarlo mediante **pacman**.

El **AUR** se creó para organizar y compartir nuevos paquetes de la comunidad y para ayudar a acelerar la inclusión de paquetes populares en el repositorio extra.

La aplicación que usaremos para instalar paquetes de los repositorios de **AUR** será **yay**.

Pasos a seguir:

Descargamos los ficheros desde **Github**:

```
cd /opt  
sudo git clone https://aur.archlinux.org/yay.git
```

Ahora cambiamos el nombre del dueño del directorio fuente:

```
sudo chown -R viriato:viriato ./yay
```

Ahora vamos al directorio y compilamos:

```
cd yay  
makepkg -si
```



Pasos a seguir:

Descargamos los ficheros desde **Github**:

```
cd /opt  
sudo git clone https://aur.archlinux.org/yay.git
```

Ahora cambiamos el nombre del dueño del directorio fuente:

```
sudo chown -R viriato:viriato ./yay
```

Ahora vamos al directorio y compilamos:

```
cd yay  
makepkg -si
```

Pasos a seguir:

Descargamos los ficheros desde **Github**:

```
cd /opt  
sudo git clone https://aur.archlinux.org/yay.git
```

Ahora cambiamos el nombre del dueño del directorio fuente:

```
sudo chown -R viriato:viriato ./yay
```

Ahora vamos al directorio y compilamos:

```
cd yay  
makepkg -si
```

Pasos a seguir:

Descargamos los ficheros desde **Github**:

```
cd /opt  
sudo git clone https://aur.archlinux.org/yay.git
```

Ahora cambiamos el nombre del dueño del directorio fuente:

```
sudo chown -R viriato:viriato ./yay
```

Ahora vamos al directorio y compilamos:

```
cd yay  
makepkg -si
```

Para instalar software solo hay que escribir **yay** en lugar de **pacman** sin necesidad de usar la orden **sudo**.



Pasos a seguir:

Descargamos los ficheros desde **Github**:

```
cd /opt  
sudo git clone https://aur.archlinux.org/yay.git
```

Ahora cambiamos el nombre del dueño del directorio fuente:

```
sudo chown -R viriato:viriato ./yay
```

Ahora vamos al directorio y compilamos:

```
cd yay  
makepkg -si
```

Para instalar software solo hay que escribir **yay** en lugar de **pacman** sin necesidad de usar la orden **sudo**.
Desde este momento podemos instalar todos los paquetes con **yay**.



Como ya podemos usar **yay**

Para instalar **GNOME**

```
yay -S gnome gdm  
systemctl enable gdm
```

Para instalar **Plasma**

```
yay -S plasma sddm  
systemctl enable gdm
```

Podemos instalar cualquier escritorio de esta manera tan sencilla.



Con esto podemos dar por terminada la instalación de Arch Linux.

En caso de dudas siempre se puede consultar su wiki en español

[https://wiki.archlinux.org/title/Main_page_\(Español\)](https://wiki.archlinux.org/title/Main_page_(Español))

Instalación de Arch Linux



Félix J. Marcelo Wirnitzer